object-oriented .NET, often without needing to manipulate a database directly or write custom SQL.

Suppose that you have an e-commerce site, and you want to load the details of a product from the database. Using low-level database access code, you'd have to open a connection to the database; write the necessary SQL with the correct table and column names; read the data over the connection; create a plain old CLR object (POCO) to hold the data; and set the properties on the object, converting the data to the correct format manually as you go. Sounds painful, right?

An ORM such as EF Core takes care of most of this work for you. It handles the connection to the database, generates the SQL, and maps data back to your POCO objects. All you need to provide is a L*INQ query* describing the data you want to retrieve.

ORMs serve as high-level abstractions over databases, so they can significantly reduce the amount of plumbing code you need to write to interact with a database. At the most basic level, they take care of mapping SQL statements to objects, and vice versa, but most ORMs take this process a step further and provide additional features.

ORMs like EF Core keep track of which properties have changed on any objects they retrieve from the database, which lets you load an object from the database by mapping it from a database table, modify it in .NET code, and then ask the ORM to update the associated record in the database. The ORM works out which properties have changed and issues update statements for the appropriate columns, saving you a bunch of work.

As is so often the case in software development, using an ORM has its drawbacks. One of the biggest advantages of ORMs is also their Achilles' heel: they hide the database from you. Sometimes this high level of